

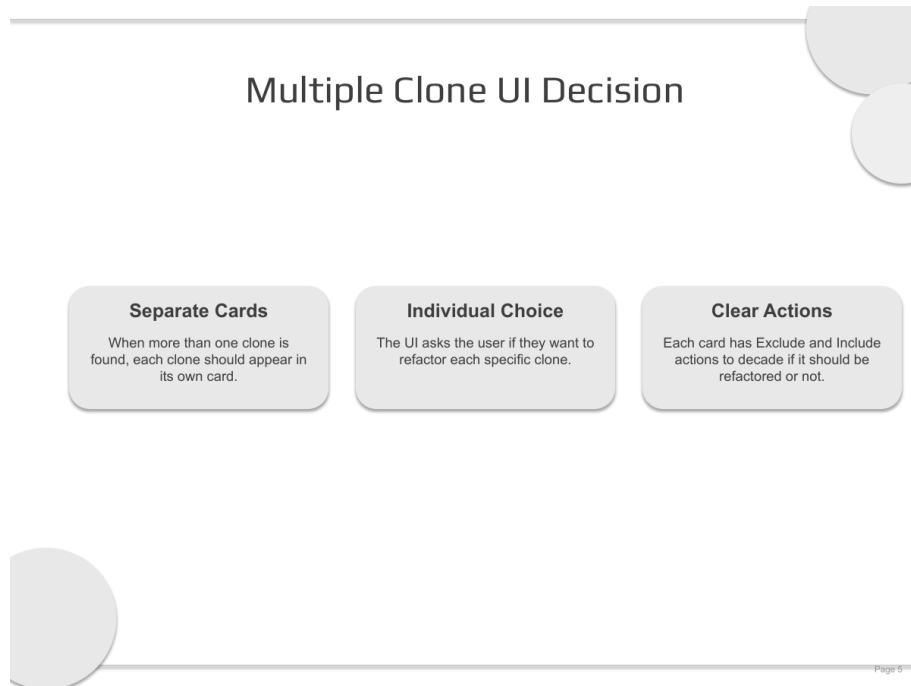
Week 4 Results

Organized visual progress from each team member's section, based on the Week 4 slides.

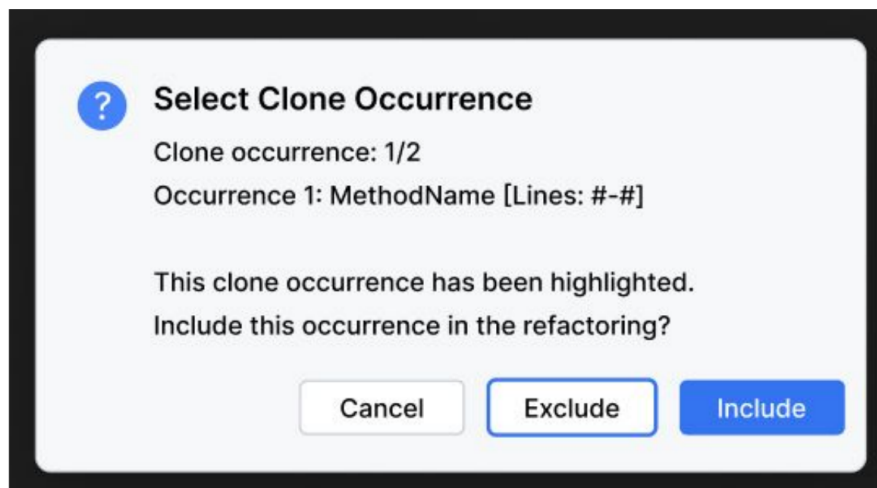
Jake Stinton — Multiple Clone UI

Progress on separate clone cards and individual choice for each clone occurrence.

Slide image 5



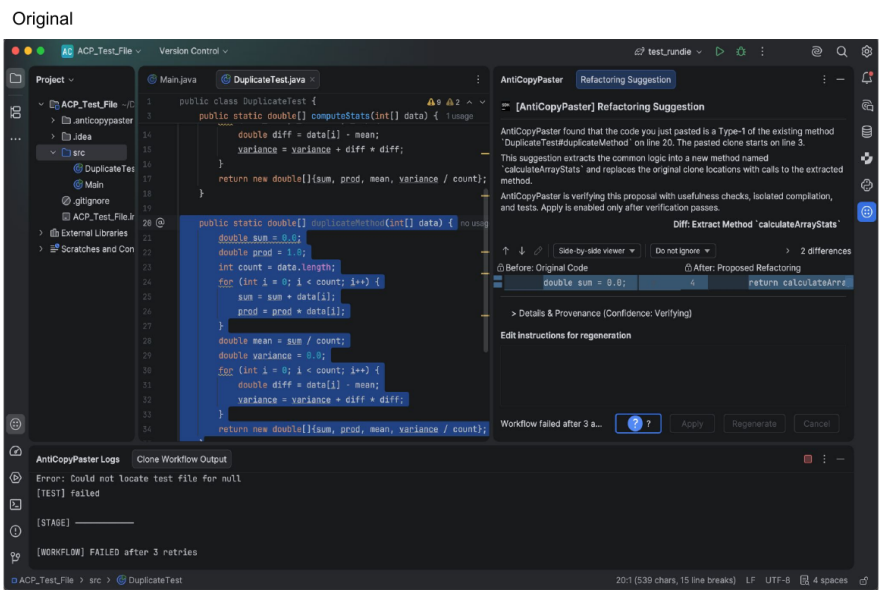
Slide image 6



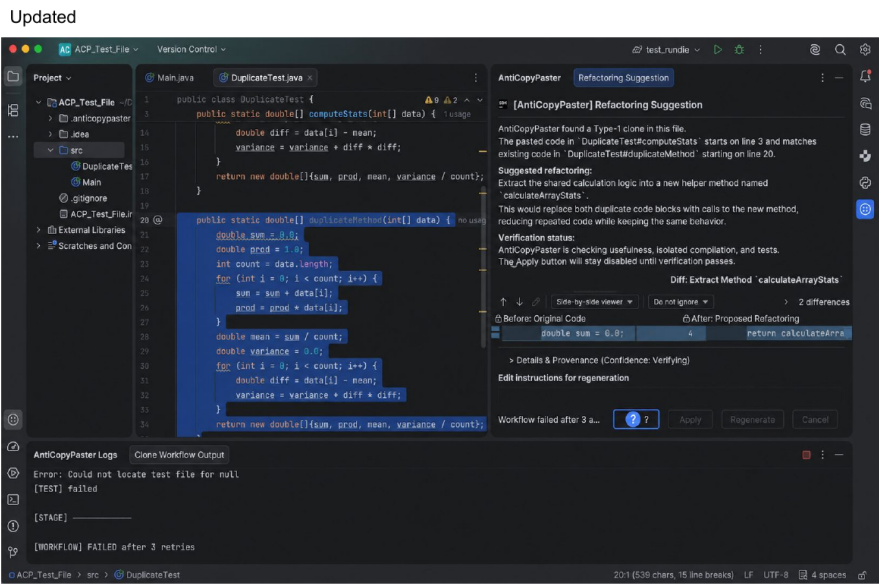
Amaro da Luz — Explanation Section

Progress on improving the explanation and details/provenance text to make clone information clearer and easier to read.

Slide image 8

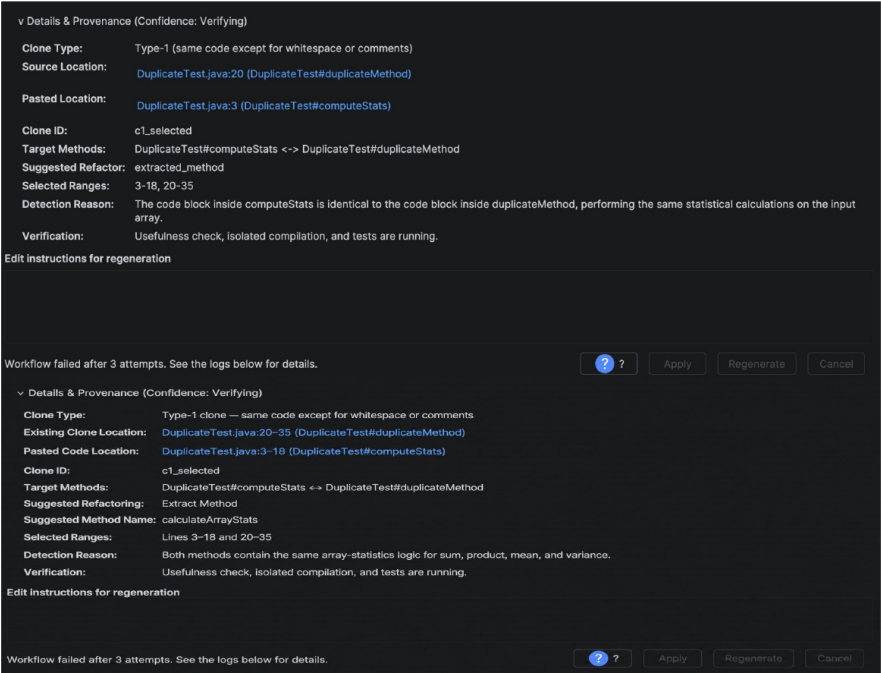


Slide image 9

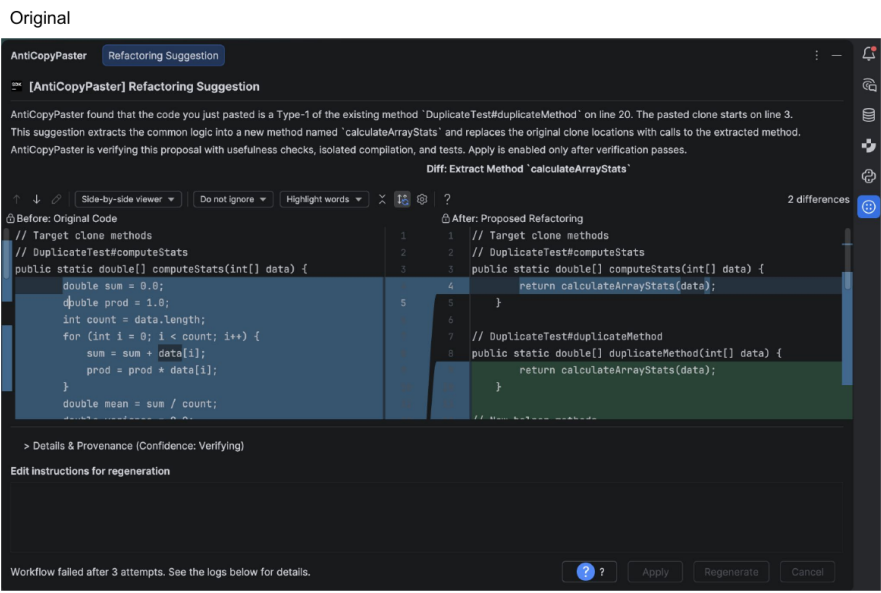


Amaro da Luz — Explanation Section (continued)

Slide image 10



Slide image 11



Amaro da Luz — Explanation Section (continued)

Slide image 12

Updated

AntiCopyPasterRefactoring Suggestion

[AntiCopyPaster] Refactoring Suggestion

AntiCopyPaster found a Type-1 clone in this file.
The pasted code in 'DuplicateTest#computeStats' starts on line 3 and matches existing code in 'DuplicateTest#duplicateMethod' starting on line 20.
Suggested refactoring:
Extract the shared calculation logic into a new helper method named 'calculateArrayStats'.
This would replace both duplicate code blocks with calls to the new method, reducing repeated code while keeping the same behavior.
Verification status:
AntiCopyPaster is checking usefulness, isolated compilation, and tests. The Apply button will stay disabled until verification passes.

Diff: Extract Method 'calculateArrayStats'

2 differences

Before: Original Code

After: Proposed Refactoring

// Target clone methods
// DuplicateTest#computeStats
public static double[] computeStats(int[] data) {
 double sum = 0.0;
 double prod = 1.0;
 int count = data.length;
 for (int i = 0; i < count; i++) {
 sum = sum + data[i];
 prod = prod * data[i];
 }
 double mean = sum / count;
 double median = ...;
}

1 // Target clone methods
2 // DuplicateTest#computeStats
3 public static double[] computeStats(int[] data) {
4 return calculateArrayStats(data);
5 }
6
7 // DuplicateTest#duplicateMethod
8 public static double[] duplicateMethod(int[] data) {
9 return calculateArrayStats(data);
10 }
11
12 // New helper method

> Details & Provenance (Confidence: Verifying)

Edit instructions for regeneration

Workflow failed after 3 attempts. See the logs below for details.

Apply

Regenerate

Cancel

Juliana — Diff Preview

Progress on the diff preview section showing original duplicate code and the extracted replacement.

Slide image 13

Diff Preview

```
❗ UserValidator.java: 42-48 ×  
42 if (u.getUsername() == null || u.getUsername().trim().is  
43     throw new IllegalArgumentException("Username is requi  
44 }
```

```
✅ validateInputs(...) ×  
12 private void validateInputs(User u) {  
13     if (u.getUsername() == null || u.getUsername().trim().  
14         throw new IllegalArgumentException("Username is requ  
15 }
```

Dhir Rana — AI Trust / Explainability

Progress on the AI Trust / Explainability view and validation breakdown.

Slide image 14

AI Trust / Explainability: Learn More View

AI Trust / Explainability — Learn More View

This detailed panel opens AFTER the user clicks "Learn more" from the compact AI Trust section.

1. DETAILED "LEARN MORE" PANEL — STATES

A SUCCESS STATE (All Validations Passed)

AI Trust / Explainability High Trust

Overall AI Trust Score: 95%

High confidence. Safe to apply.

Formula: 20% Clone Detection Confidence + 20% Refactoring Agent Confidence + 20% Refactoring Usefulness + 20% Compilation Result + 20% Test / Behavior Preservation

Rationale: All validations passed with strong confidence. The refactored code complies successfully and preserves original behavior in tests.

What this means: High trust in this change. Safe to apply after a quick review of the diff and explanation.

A WARNING / PARTIAL STATE (Some Validations Have Issues)

AI Trust / Explainability Warning: Trust

Overall AI Trust Score: 65%

Some checks failed. Review before applying.

Formula: 20% Clone Detection Confidence + 20% Refactoring Agent Confidence + 20% Refactoring Usefulness + 20% Compilation Result + 20% Test / Behavior Preservation

Rationale: Some validations scored below the threshold. Specifically, usefulness is moderate and test preservation is low.

What this means: Review the diff and explanation carefully. Consider manual verification before applying this change.

A FAILURE / UNAVAILABLE STATE (Score Cannot Be Calculated)

AI Trust / Explainability Unavailable

Overall AI Trust Score: Unavailable

Score cannot be calculated.

Formula: 20% Clone Detection Confidence + 20% Refactoring Agent Confidence + 20% Refactoring Usefulness + 20% Compilation Result + 20% Test / Behavior Preservation

Reason: Compilation failed due to syntax errors in the refactored code. The score cannot be calculated until this step is resolved.

What to do next: Fix the compilation issues and re-run validation to see the updated trust score and breakdown.

2. IN-CONTEXT USAGE EXAMPLE

[AntiCopyPaste] Refactoring Suggestion

1. Explanation
AntiCopyPaste detected that the code you pasted is a Type-2 clone of the existing method in the same class.
• Existing clone location: UserValidator.java:43-48
• Pasted clone location: UserValidator.java:101-107
• Suggested method name: validateEmail()
The assistant suggests extracting the common validation logic into a new method and replacing the duplicate with a method call.

2. Diff Preview (Preview)

```
Original (Duplicate Code)
if (isPhoneNumber()) {
    validatePhoneNo();
} else if (isEmail()) {
    validateEmail();
} else {
    throw new IllegalArgumentException("Invalid input");
}

Refactored (Replacement)
if (isPhoneNumber()) {
    validatePhoneNo();
} else {
    validateEmail();
}
// ...
// New method in Validator.java
private void validateEmail() {
    // ...
}
```

3. AI Trust / Validation Learn more →

95%
AI Trust Score

Formula: 20% each across five workflow stages.

4. Actions

Apply Reject Preview Diff ...

5. Clicking "Learn more" opens the detailed AI Trust / Explainability view (this panel).

Fin Naughton — Buttons / Actions

Progress on action buttons, including apply/reject/details behavior and disabled states when validation fails.

Slide image 15

Buttons

Individual Acceptance

The user should have the opportunity to apply, reject, or view details for each clone instance

Disabled Acceptance

The apply button should be disabled in the case that validation fails, as there will be nothing to apply.

Original

Before: Original Code

double sum = 0.0;

After: Proposed Refactoring

4

return calculateArr

> Details & Provenance (Confidence: Verifying)

Edit instructions for regeneration

Workflow failed after 3 a...

?

Apply

Regenerate

Cancel

Slide image 16

v Details & Provenance (Confidence: Verifying)

Clone Type:

Type-1 (same code except for whitespace or comments)

Source Location:

DuplicateTest.java:20 (DuplicateTest#duplicateMethod)

Pasted Location:

DuplicateTest.java:3 (DuplicateTest#computeStats)

Clone ID:

c1_selected

Target Methods:

DuplicateTest#computeStats <-> DuplicateTest#duplicateMethod

Suggested Refactor:

extracted_method

Selected Ranges:

3-18, 20-35

Detection Reason:

The code block inside computeStats is identical to the code block inside duplicateMethod, performing the same statistical calculations on the input array.

Verification:

Usefulness check, isolated compilation, and tests are running.

Edit instructions for regeneration

Workflow successful.

Apply

Reject

Details

Editor View

v Details & Provenance (Confidence: Verifying)

Clone Type:

Type-1 (same code except for whitespace or comments)

Source Location:

DuplicateTest.java:20 (DuplicateTest#duplicateMethod)

Pasted Location:

DuplicateTest.java:3 (DuplicateTest#computeStats)

Clone ID:

c1_selected

Target Methods:

DuplicateTest#computeStats <-> DuplicateTest#duplicateMethod

Suggested Refactor:

extracted_method

Selected Ranges:

3-18, 20-35

Detection Reason:

The code block inside computeStats is identical to the code block inside duplicateMethod, performing the same statistical calculations on the input array.

Verification:

Usefulness check, isolated compilation, and tests are running.

Edit instructions for regeneration

Workflow unsuccessful.

Apply

Reject

Details

Editor View

Next Steps

Slide showing the GitHub workflow and next steps for implementation and team updates.

Slide image 17

